AI 原生编程系统
完整设计构想与工程架构说明

从 C 语言与传统工具链出发，构建受控的智能软件生产体系

核心主张：传统代码是骨架，AI 是细胞；人类负责系统结构，AI 负责局部生产，传统编译器与验证工具负责检查，TCT – ECL 负责结构边界和持续工程控制。

文档性质：基于讨论内容整理的设计稿。文中语法、交互和架构均为概念设计，不代表已有标准、已实现产品或已经验证的技术结论。

版本：讨论汇编版｜2026 年 10 月

目录

01 设计目标与总体哲学
02 语言基础：以 C 生态为底座
03 函数成为完整的工程对象
04 AI 上下文：作用域、权限与生命周期
05 AI::EdgeFun：把边界转换变成一等单元
06 AI 原生 IDE：统一编辑区与命令模型
07 键盘、鼠标、双屏与注意力管理
08 执行图、任务图与未完成工作
09 函数生产、测试、调试和交付生命周期
10 TCT：翻译、传输与计算的结构控制
11 ECL：贯穿工程生命周期的控制循环
12 复杂度扩散：AI 生产力增长后的主要风险
13 整体系统架构与职责分层
14 未来计算机教育与开发者能力
15 设计原则、边界与待验证问题
附录 A 概念语法与交互示例
附录 B 术语表与核心结论

阅读方式：前半部分描述系统组件，后半部分解释这些组件如何协作。TCT 是结构组织原则；ECL 是反馈控制机制；IDE 是人类操作入口；编译器和运行时仍是确定性执行基础。

01 | 设计目标与总体哲学

这套构想不是在传统 IDE
旁边加一个聊天窗口，也不是把自然语言简单转换成代码补全。它试图重新组织软件生产的基本单元、开发者与 AI 的交互方式、AI 可见信息的范围、任务生命周期，以及工程复杂度的传播路径。

1.1 核心目标

- 让 AI 能够大规模生产局部软件实现，同时不让系统复杂度失控。
- 让每个生成任务拥有明确的意图、上下文、权限、接口、测试和历史。
- 让开发者把注意力放在架构、接口、关键算法和验收上，而不是重复手写常规函数。
- 保留传统编译器、链接器、调试器和操作系统生态提供的确定性与可观察性。
- 让工程能够持续观察、验证、回退和演进，而不是依赖一次性生成结果。

1.2 总体哲学

> 让需求清晰，让传输受控，让计算纯粹，让复杂度局部化，让工程始终可观察、可验证、可管理。

AI 提高的是实现生产速度，不会自动降低系统复杂度。若缺少约束，生成速度越快，依赖、接口变化、错误传播和并行冲突也可能扩散得越快。因此，系统必须同时设计生产能力与复杂度控制能力。

1.3 人、AI 与传统工具的分工

主体	主要职责	不应被混淆的边界
人类开发者	定义意图、架构、关键接口、关键算法、约束与验收标准	不必亲自实现每个常规函数，但必须理解系统结构
AI	在明确范围内生产函数、结构体、适配器、测试、文档与局部修改	不能默认拥有全项目读写权，也不能把生成结果当成正确性
编译器与工具链	编译、链接、类型检查、调试、静态分析和确定性执行	不负责理解模糊需求，也不替代系统设计
TCT – ECL	管理复杂度的结构传播与工程反馈	不是某个 AI 模型，也不是单纯的代码生成流水线

02｜语言基础：以 C 生态为底座

设计倾向是以 C 语言及其成熟工具链为基础，而不是从零发明一套庞大语言。C 的价值不只在语法，而在于它与编译器、链接器、ABI、操作系统、调试器和大量既有软件之间形成了长期稳定的工程接口。

2.1 优先复用的基础

- 结构体、函数、指针与类型系统。
- 头文件与源文件的声明/实现分离。
- 编译器、链接器、ABI 和成熟的构建流程。
- 调试器、静态分析器、性能分析器及操作系统生态。

2.2 AI 生成结果的落点

AI 生成的实现最终应尽量落成普通 C 函数、结构体和可编译代码。它们能够像普通代码一样被调用、跳转定义、修改、设置断点、单步调试和版本管理。生成结果一旦接受并锁定，就不应依赖运行时 AI 才能执行。

2.3 最小语言扩展原则

- 尽可能少增加语言关键字。
- 能由 IDE、标准库、元数据或既有语法完成的能力，不必强行塞进语言核心。
- 概念语法先作为 IDE 指令或元数据表达，只有在实践证明必要后，才考虑成为正式语言语法。
- AI 可以跨语言生成 C、Rust、Python 等实现，但必须明确处理内存管理、错误模型、整数溢出、并发语义和资源所有权等差异。

> 关键判断：AI 原生不等于抛弃传统编程语言。更合理的路径是保留成熟的确定性执行底座，在其上重构函数生产与工程组织方式。

2.4 设计张力

C 提供直接、明确、低层的控制，但也把内存安全、资源管理和接口纪律更多地交给开发者。AI 生成代码时，必须把这些约束显式化；不能因为生成速度提高，就忽略指针有效性、生命周期、缓冲区边界、错误路径和并发访问。

03 | 函数成为完整的工程对象

传统函数通常被看作声明与实现。新构想把函数提升为一个可管理、可追踪、可验证的工程对象。函数不仅有代码，还携带其意图、依赖、契约、测试、生成记录和验证状态。

3.1 函数对象的组成

组成	含义
Identity	函数名称、稳定身份与所属模块
Intent	自然语言描述：函数要解决什么问题、为何存在
Context	AI 可见的代码、依赖、文档、约束与权限
Contract	输入、输出、类型、错误语义、前置/后置条件与行为约束
Implementation	实际实现代码
Tests	测试接口、测试数据、测试代码及覆盖场景
History	生成记录、模型/提示版本、人工修改、验证结果与接受记录

3.2 自然语言声明的概念

开发者可以用一句意图描述创建函数或结构体，由 IDE
生成相应的声明、实现骨架、测试和文档。示例语法仅用于表达交互方向，并非 C 标准语法：

```
! ■■■■■■■■■■■■■ gcd()\n\n! ■■■■■■■■■■■■■■■■■ID■■■■■■■ User
```

讨论中提出一种简洁约定：末尾带 `()` 表示函数，不带 `()` 表示结构体。IDE
可将自然语言指令转化为明确的类型和函数对象。

```
AI::Fun("■■■■■■■■■■■■■") gcd;\nAI::Struct("■■■■■■■■■■■■■■■■■ID■■■■■■■") User;
```

3.3 人类与 AI 的生产分工

- 人类重点负责系统架构、核心算法、关键接口、约束条件和验收。
- AI 负责常规业务函数、数据转换、边界适配器、测试、文档和局部重构。
- 极短、确定、容易直接表达的逻辑可以人工编写，也可以交给 AI；不必把 “ 所有函数都必须 AI 生成 ” 变成教条。
- 生成函数必须可以回到普通代码层面检查、修改、调试和锁定。

3.4 设计收益与代价

收益是意图、代码、测试和历史不再分散，函数能作为一个完整单元管理。代价是 IDE 需要维护额外元数据、契约格式、身份稳定性和历史关联；如果元数据过重，也可能变成新的复杂度来源。因此元数据应保持轻量、可读、可迁移，并允许脱离特定 IDE 使用代码本身。

04 | AI 上下文：作用域、权限与生命周期

AI 不应默认读取整个项目。上下文应像变量作用域、内存所有权和模块权限一样受到约束。它不仅决定模型“知道什么”，还决定模型“允许修改什么”以及任务结束后哪些信息应当释放。

4.1 分层上下文

层级	范围	典型用途
L0	当前函数	解释、修复或生成当前函数
L1	当前文件与直接依赖	理解函数调用、类型定义和局部数据流
L2	当前模块	跨文件接口、模块内部协作与局部重构
L3	整个项目	架构级分析、全局依赖调整；需要明确授权

4.2 默认规则

- 默认只读取当前任务所需的信息，而不是全项目灌入上下文。
- 默认只修改任务明确指定的范围。
- 依赖信息不足时，AI 申请扩大可见范围，而不是自行扫描和修改。
- 上下文超过预算时，暂停并请求扩大预算或缩小任务。
- 记录必要的生成历史和决策依据，临时上下文在任务结束后释放或归档。
- 可维护轻量级全局符号索引、依赖索引和语义索引，但索引存在不等于每次任务都加载全部项目内容。

4.3 上下文声明示例

```
AI Context:\n    Scope: loadConfig\n    Inputs: Config.h, File.h\n    Outputs: Config\n    Permissions: ReadFil
```

4.4 与传统编程概念的对应

传统概念	AI 工程中的对应概念
变量生命周期	AI 上下文生命周期
内存所有权	上下文访问权限与修改权限
函数作用域	AI 任务作用域
模块依赖	AI 可见依赖
资源预算	Token、推理时间与计算预算
内存隔离	上下文隔离与任务隔离
函数返回	生成、验证、交付并结束任务

4.5 必须处理的边界问题

上下文系统还需要解决信息过期、索引与真实代码不一致、跨任务共享信息、敏感数据隔离、并行修改冲突和上下文注入等问题。权限应按读取、创建、修改、删除、执行、提交等动作分别定义，而不是只有一个笼统的“允许 AI 操作项目”开关。

05 | AI::EdgeFun：把边界转换变成一等单元

AI::EdgeFun 是讨论中提出的一个重要函数类别：专门负责隔离外部系统与内部业务模型之间的变化。它与 Adapter Pattern、Anti-Corruption Layer 和 Hexagonal Architecture 有相通之处，但在本构想中，它被纳入 AI 函数生产与验证体系。

5.1 典型用途

- 外部 API 字段与内部结构体之间的转换。
- 数据库 schema 与业务对象之间的映射。
- 新旧接口兼容和版本迁移。
- 错误码、状态码和异常语义映射。
- 单位、时区、编码格式和协议转换。
- 外部支付、身份认证、消息系统等服务的边界隔离。

5.2 概念示例

```
AI::EdgeFun(\n    "■■■■■■■■■■■■■■■■ Payment ■■■\n    ■■■■■■■■■■■■■■■■■■■■"\n) paymentAdapter;
```

5.3 设计约束

- 外部字段名、协议细节和供应商特性尽量止步于边界层。
- 内部业务代码依赖稳定的内部类型和契约，而不是外部 API 的原始结构。
- 转换失败、缺失字段、未知枚举值和版本不兼容必须有显式策略。
- 边界函数应有独立测试，覆盖正常输入、异常输入、缺失字段和未来字段。

> EdgeFun 的核心价值不是多增加一层包装，而是把变化集中在一个明确、可测试、可替换的位置，避免外部复杂度污染核心计算。

06 | AI 原生 IDE：统一编辑区与命令模型

用户不希望固定 AI 聊天侧栏或常驻 AI 窗口。理想交互是：编辑器仍然是主要工作空间，AI 默认隐身；开发者在需要时从当前编辑位置唤醒 AI，输入指令，结果就地呈现，完成后恢复原有编辑状态。

6.1 交互原则

- 默认处于普通代码编辑状态，AI 不闪烁、不自动弹建议、不持续争夺焦点。
- 按下 AI 快捷键后，在当前光标位置出现临时命令输入层。
- 回车执行，回答或生成结果在当前位置或局部临时区域显示。
- 任务完成后命令层收起；焦点根据操作类型返回代码编辑区或保留在结果上。
- 命令自动进入命令历史；耗时任务可以后台运行、取消、回退。
- 只有需要开发者决策、授权或确认时，AI 才主动打断。

6.2 符号命令体系

符号	语义定位	典型用途
?	询问过去：查找已有知识与代码	查找函数、引用、历史实现和依赖
/	解释当下：理解当前对象	解释当前函数、数据流和调用路径
!	书写未来：生产新对象	创建函数、结构体、修改实现或生成测试
.	存档动作	Git 存档或提交操作；需避免普通句号误触发
*	IDE 操作命令	新建/切换文件、跳转、编译、运行、查找引用

6.3 示例

```
? ■■■■■■■■■■\n/ ■■■■■ loginUser() ■■■■■■\n! ■■■■■■■■■■■■■\n* ■■■■■ User.c\n* ■■■■ User.c\n* ■■■■ userSave(
```

6.4 命令执行分层

层级	命令类型	执行方式
L0	确定性操作：新建、切换、跳转、编译、运行、保存	由 IDE 本地确定性执行，不需要 AI 推理
L1	语义操作：查找、解释、定位依赖	优先使用符号索引和语义检索；必要时由小模型辅助
L2	生产操作：生成函数、结构体、修改模块、生成测试	需要上下文权限、范围确认、编译测试和回退机制

6.5 命令历史与 Git 历史

命令历史记录用户问了什么、AI 给出什么建议、用户选择了什么、哪些生成被接受或拒绝。Git 历史记录实际工程文件和代码变化。二者可以互相引用，但不应混成一套记录：一次命令不一定产生代码变化，一次代码变化也不一定来自 AI 命令。

07 | 键盘、鼠标、双屏与注意力管理

7.1 键盘模式

开发者按下 AI 快捷键，在当前行或光标位置输入自然语言命令，回车执行。命令输入层是临时的，不需要切换到一个独立聊天窗口。

7.2 鼠标模式

不强迫所有人采用纯键盘工作流。编辑器左侧行号旁可以提供 AI 功能符号，类似断点按钮。点击后在对应行附近出现需求输入区；用户输入后回车，或点击确认标记接受结果。它适合新手、鼠标习惯用户，以及需要明确选择代码位置的场景。

7.3 双屏工作方式

双屏不只是把两个传统代码窗口并排放置。比如左屏编辑 User.c，右屏编辑 Database.c；两边都可以独立提问、生成、测试和调试。两个编辑区拥有各自的工作焦点，但共享工程状态、符号索引和依赖信息。

- 并行 AI 任务必须记录各自的目标文件、接口版本和依赖快照。
- 两个任务若修改同一接口，应触发冲突检测或显式协调。
- 一侧生成的新接口可能影响另一侧任务，不能让两个 AI 在彼此不知情的情况下继续基于过期假设工作。

7.4 注意力单通道

> 软件内部可以同时有很多任务，但开发者同一时刻只需要关注一个主要交互位置。

AI 默认隐身、主动唤醒、局部呈现结果；不要用多个侧栏、弹窗、自动补全和任务通知同时争夺注意力。后台任务可以并行，但前台交互应保持单一主焦点。

7.5 交互一致性

键盘与鼠标只是同一套命令和任务系统的两种入口。命令解析、权限、上下文、任务状态、历史记录和验证流程应共享，避免形成两套行为不一致的产品逻辑。

08 | 执行图、任务图与未完成工作

传统 IDE 主要展示程序运行时的调用关系，但 AI
原生开发还需要管理开发过程中的未完成工作。为此需要区分执行图与任务图。

8.1 执行图

执行图描述程序运行时实际调用了什么函数、经过哪些模块、发生了哪些数据流和副作用。它属于运行时行为视图。

8.2 任务图

任务图描述工程开发过程中哪些函数待实现、待检查、待测试、被阻塞或已经完成。它属于开发过程视图。

```
int main(void)\n{\n    User *user = userLoad(1001);\n    ! userSave(user);\n    return 0;\n}
```

如果 `userSave()` 尚未实现，IDE
可以把它登记为待办任务，而不是强制打断当前代码编写。该标记表达的是开发任务，不是普通 C 语法。

8.3 任务依赖示例

```
main\n ■■■ userSave\n      ■■■ userValidate\n      ■■■ jsonEncode\n      ■■■ fileWrite
```

8.4 任务状态

状态	含义
New	新发现或刚创建，尚未规划
Planned	已明确目标与依赖，等待执行
Blocked	被缺失接口、依赖或决策阻塞
Ready	前置条件满足，可以执行
Testing	实现已产生，正在验证
Done	达到当前验收条件
Cancelled	任务被取消，保留取消原因与历史

8.5 任务图与执行图的关系

任务图中的一个节点可以对应执行图中的一个函数或模块，但二者不能等同。一个函数可能已经存在却没有测试任务；一个任务也可能横跨多个函数。任务图应记录目标、依赖、状态、负责人/代理、版本和验收条件，并与代码实体建立稳定关联。

09 | 函数生产、测试、调试和交付生命周期

函数生产不是“ 生成代码然后结束 ”。完整生命周期必须把需求、上下文、生成、编译、测试、审查、接受、锁定和历史记录连接起来。

9.1 标准生产流程

- 人类提出需求，说明意图、边界、约束和期望行为。
- IDE 建立局部 AI 上下文，确定可见依赖、读写范围和资源预算。
- AI 检查已有函数、类型、接口和依赖，避免重复造轮子。
- AI 生成函数声明与实现，并说明关键假设。
- AI 同时生成测试接口、测试数据和测试代码。
- 传统编译器执行编译与链接；静态分析器检查指定类别的问题。
- 运行自动测试；必要时执行性能、内存和边界场景检查。
- 人工审查关键逻辑、接口契约和测试充分性。
- 接受后锁定生成版本，记录来源、修改和验证状态。
- 归档必要上下文与任务结果，释放临时上下文。

9.2 测试 UI 与调试器联动

调试器可以在程序停留于某个位置时，把当前运行时数据提供给测试 UI，用于快速构造函数测试和复现问题。但必须隔离副作用：测试不能随意污染正在运行的进程、数据库或外部服务。应优先使用快照、复制数据、沙箱、事务回滚或模拟对象。

9.3 验证结果不能混为一谈

结果	实际能说明什么	不能说明什么
编译成功	语法、类型和链接等指定阶段通过	不能证明业务行为正确
测试成功	已覆盖测试场景通过	不能证明未覆盖场景无错误
静态分析通过	指定规则或缺陷类别未被工具发现	不能证明不存在所有缺陷
形式化验证	在明确模型、前提和假设下证明了特定性质	不能自动证明模型与现实需求完全一致
生产可靠	还需要真实运行数据、监控、故障响应和长期维护证据	不是一次编译或测试就能得到的标签

9.4 版本锁定与可追溯性

接受的 AI 生成结果应转化为可审查、可版本控制的普通代码。记录至少应包含生成目标、上下文范围、模型/工具版本、关键提示或契约、代码差异、测试结果、人工修改和接受者决策。应允许开发者关闭
AI、导出代码并继续使用传统工具链。

10｜TCT：翻译、传输与计算的结构控制

TCT 是 Translation – Transmission – Computation 的缩写。它不是某个 AI 功能，而是组织需求、信息和计算在工程中如何分层、如何传递、如何避免互相污染的结构原则。

10.1 T1：Translation｜翻译

把人类意图转化为明确的函数、接口、环境、对象、方法、目的和约束。输出应尽可能成为机器可检查的契约，而不是只留下一句模糊自然语言。

- 明确主体：谁或什么对象在执行。
- 明确环境：函数在哪种运行条件下工作。
- 明确输入输出、错误语义和副作用。
- 明确成功条件、失败条件和边界条件。
- 把模糊需求转成可讨论、可修改、可验证的结构。

10.2 T2：Transmission｜传输

管理外部输入、UI、API、消息、协议、文件和数据转换。使用 EdgeFun、Adapter 和明确接口隔离外部变化，避免外部系统的复杂性直接进入核心业务逻辑。

- 输入来源和数据格式要显式。
- 协议转换、版本兼容和错误映射集中在边界层。
- 传输过程不能悄悄改变业务语义。
- 边界失败要能被定位、记录和测试。

10.3 T3：Computation｜计算

保持核心计算清晰、局部、可预测。尽可能把计算与外部副作用分离，控制内存、执行顺序、延迟和资源。

- 计算逻辑尽量依赖明确输入，不暗中读取全局状态。
- 副作用应通过明确接口暴露。
- 优先考虑可测试性、可复现性和资源可预测性。
- 对实时或低延迟场景，关注尾延迟、内存分配、锁竞争和不可预测运行时行为，而不仅是吞吐量。

> TCT 的总纲：让需求不产生歧义，让传输不干扰业务，让计算保持纯粹。

10.4 TCT 与既有工程思想

TCT 与分层架构、端口适配器、领域边界、纯函数和数据管道等思想存在交集。它的价值取决于能否形成更清楚的工程边界和更可执行的检查规则，而不是仅仅换一套术语。它应当帮助开发者识别复杂度从哪里进入、经过什么接口、在哪里被转换，以及是否污染核心计算。

11｜ECL：贯穿工程生命周期的控制循环

ECL 是 Engineering Control Loop。它负责工程从设计、生成、编译、测试、运行到维护的持续反馈。它不是一次性的测试步骤，而是贯穿整个工程生命周期的观察—决策—执行—验证循环。

11.1 基本循环

阶段	工作	可能使用的信号
Observe｜观察	观察工程结构、代码差异、任务状态、运行状态和异常	编译日志、测试结果、性能指标、崩溃、依赖变化
Decide｜决策	判断偏差、风险、优先级和修改目标	需求契约、验收条件、风险等级、历史记录
Act｜执行	生成、修改、重构、编译、测试或部署	AI 代理、IDE 命令、传统构建与部署工具
Verify｜验证	确认行动是否达到目标，是否引入副作用	测试、静态分析、回归、人工审查、运行监测

11.2 AI 在 ECL 中的位置

AI 可以参与观察、分析和执行，但必须有清晰权限与验收标准。AI 可以提出决策建议、生成修改方案或执行获准操作；高风险操作仍需人工确认。系统要保留“谁观察到什么、基于什么判断、执行了什么、验证结果如何”的可追溯链条。

11.3 ECL 与 TCT 的关系

TCT 控制复杂度如何在工程结构中传递；ECL 观察复杂度和行为变化，并通过反馈进行纠偏。TCT 更偏结构约束，ECL 更偏动态控制。它们不是简单的串行阶段，也不是只能单向运行的流水线，而是相互作用的两类机制。

> 可以把 TCT 理解为系统的结构与边界纪律，把 ECL 理解为系统持续检查自身并调整的反馈机制。

12 | 复杂度扩散：AI 生产力增长后的主要风险

AI 能快速生成局部代码，但系统复杂度并不会因为生成速度变快而自动消失。相反，若验证、接口管理和依赖控制跟不上，复杂度可能扩散得更快。

12.1 典型扩散链



12.2 复杂度控制原则

- 复杂度尽可能局部化，不让局部实现的细节泄漏到其他模块。
- 模块之间使用明确、稳定、可检查的接口。
- 外部变化由边界函数隔离。
- 局部故障不应轻易扩散为全局故障。
- 优先消除不确定性，而不是用更多抽象把不确定性藏起来。
- 避免无必要的依赖、框架和层次。
- AI 上下文不能无限扩张；权限和资源预算必须显式。
- 每次修改都应当可以观察、验证和回退。
- 生成速度不能长期超过验证能力。
- 架构变化必须显式管理，不能把结构性变化伪装成普通函数修改。

12.3 既有工程取向

讨论中反复强调复杂度 containment、局部失败与明确接口；强调小实现、高功能密度、根因分析和可预测性。对实时、单机、单线程或低延迟系统，常常优先考虑延迟和可预测性，而非只追求吞吐量；避免不必要的 GC、死锁、过度 RPC、过重生态和企业式抽象。

曾提出一种工程资源投入的直觉表达：生产能力 1×、冗余能力 2×、消除不确定性的投入 10×。它是强调风险消除价值的设计口号，不是经过统计验证的通用成本定律。

12.4 事故驱动开发

事故和真实失败可以揭示隐式假设、薄弱边界与未覆盖路径。将事故转化为测试、约束、监控和接口改进，可以让工程系统逐步提高可控性。事故驱动不意味着等待事故发生，而是把已经发生的故障和近失事件转化为可复用的工程知识。

13 | 整体系统架构与职责分层

整体架构不是一条严格单向流水线，而是由人类意图、IDE 交互、AI 生产、上下文权限、TCT 结构控制、传统工具链和 ECL 反馈循环共同组成。



13.1 各层职责

层/组件	核心职责	关键输出
C 与传统编译器	提供确定性语言表达、编译、链接与执行基础	普通可编译代码与运行产物
AI 原生函数	生产局部实现与配套测试	函数、结构体、EdgeFun、测试与文档
上下文管理	限制 AI 可见范围、修改范围和资源使用	上下文快照、权限与预算记录
IDE	提供人类与系统之间统一、低干扰的交互入口	命令、任务、编辑和审查体验
任务图	组织尚未完成的工程工作	依赖、状态、阻塞原因和验收条件
TCT	控制复杂度的结构传播路径	明确的翻译、传输与计算边界
ECL	持续观察并反馈工程状态	偏差、行动、验证和演进记录

13.2 一次任务如何穿过系统

- 用户在当前函数附近输入 `!` 指令。
- IDE 解析命令，建立任务节点和局部上下文。
- 上下文管理器检查权限、依赖与预算。
- T1 将自然语言需求转成函数契约；T2 确认外部数据边界；T3 确认核心计算职责。
- AI 生成实现和测试；编译器、静态分析和测试工具执行验证。
- 开发者审查并接受或拒绝；接受结果进入版本历史。
- ECL 记录观察信号与验证结果，必要时反馈修改接口、任务或上下文。

14 | 未来计算机教育与开发者能力

当 AI 可以承担大量局部实现，计算机教育不应只训练学生快速手写代码，也应训练他们理解系统结构、接口、验证、上下文和复杂度传播。

14.1 课程能力版图

- 编程语言、算法与数据结构：理解计算本身及其成本。
- 编译器、操作系统与运行时：理解代码如何变成实际执行。
- AI 上下文与作用域：理解模型看到什么、能修改什么、何时结束任务。
- 函数意图与行为契约：把需求转成明确、可验证的接口。
- AI 函数生产生命周期：生成、审查、测试、接受、锁定和追溯。
- 自动测试、静态分析与形式化验证：理解不同验证手段的能力边界。
- 任务图与工程依赖：组织未完成工作和跨模块协作。
- TCT 工程结构：管理翻译、传输和计算的边界。
- ECL 反馈控制：观察系统、识别偏差、执行修正并验证。
- 复杂度扩散与故障隔离：理解局部变化如何影响整个系统。

14.2 开发者角色变化

未来程序员未必需要亲自实现每一个常规函数，但必须能判断函数是否属于正确的模块、契约是否清楚、依赖是否合理、测试是否覆盖关键行为、失败是否会扩散，以及系统是否仍然可维护。

14.3 教育重点

重点不是把 AI 当作自动答案机，而是训练学生把 AI
当作受控的生产代理：能拆分任务、提供必要上下文、设定权限、评估结果、验证行为并承担系统层面的判断责任。

15 | 设计原则、边界与待验证问题

15.1 已形成的设计主张

- AI 应成为软件生产体系的一部分，而不是编辑器旁边的聊天插件。
- 函数应当是包含意图、契约、实现、测试和历史的完整工程对象。
- AI 上下文必须具有作用域、权限、预算和生命周期。
- EdgeFun 用于隔离外部系统变化，稳定内部业务接口。
- IDE 应以当前编辑区为中心，默认低干扰，按需唤醒 AI。
- 键盘和鼠标应是同一交互系统的两种入口；双屏任务需要共享工程状态并管理冲突。
- 执行图与任务图应分开，分别描述运行时调用和开发时工作。
- 编译、测试、静态分析、形式化验证和生产可靠性必须严格区分。
- TCT 负责结构边界；ECL 负责持续观察与反馈。
- 复杂度局部化、明确接口、故障隔离、可观察、可验证和可回退是系统底线。

15.2 仍待验证的关键问题

- 自然语言函数声明最终应采用注释/元数据、IDE 命令、宏式语法，还是正式语言扩展？
- 函数契约如何表达，才能既足够机器可检查，又不变成难以维护的重型规范？
- 上下文的最小必要范围如何自动确定？索引、检索与模型上下文如何同步？
- 多个 AI 代理并行修改同一模块时，如何进行接口版本管理、冲突检测和事务式接受？
- AI 生成测试的独立性如何保证，避免实现与测试共享同一个错误假设？
- 如何衡量上下文质量、生成质量、复杂度增长和验证成本？
- 如何保证生成代码脱离 AI 平台后仍可编译、调试、维护和迁移？
- 如何处理安全权限、密钥、隐私数据、供应链风险和恶意上下文注入？
- TCT 与 ECL 如何形成可执行的检查清单、静态规则、IDE 提示和度量指标？

15.3 设计风险

如果把所有自然语言都变成元数据，可能造成新的 DSL
复杂度；如果上下文权限设计过于僵硬，会阻碍跨模块任务；如果过度依赖 AI 生成测试，可能出现自证循环；如果 IDE 交互过于隐形，新手可能不知道系统正在做什么；如果任务图与
Git、构建系统重复记录，又可能产生多个不一致的事实来源。

15.4 建议的原型顺序

- 第一阶段：普通 C 工程中的局部函数生成、上下文限制、编译与测试。
- 第二阶段：函数元数据、任务图、命令历史和接受/回退机制。
- 第三阶段：EdgeFun、跨文件上下文申请、模块级任务和接口冲突检测。
- 第四阶段：TCT 检查规则、ECL 运行反馈、性能与故障数据整合。
- 第五阶段：双屏并行代理、项目级任务编排和更复杂的验证工作流。

> 原型原则：先证明局部闭环可靠，再扩大任务范围；不要一开始就构建一个能够任意修改整个项目的全能代理。

附录 A｜概念语法与交互示例

A.1 函数与结构体

```
! ■■■■■■■■■■■■ gcd()\n! ■■■■■■■■■■■■■■■ID■■■■■■■ User\n\nAI::Fun("■■■■■■■■■■■■■") gcd;\nAI::Struct("■■
```

解释：`!` 表示生产意图；`()` 表示函数；不带 `()` 的名称表示结构体。`AI::Fun` 与 `AI::Struct` 是概念化表达，可由 IDE 解释，不应误认为已经存在的 C 标准扩展。

A.2 EdgeFun

```
AI::EdgeFun(\n    "■■■■■■■■■■■■■■■ Payment ■■■■\n    ■■■■■■■■■■■■■■■■■■■■■"\n) paymentAdapter;
```

A.3 上下文

```
AI Context:\n    Scope: loadConfig\n    Inputs: Config.h, File.h\n    Outputs: Config\n    Permissions: ReadFil
```

A.4 IDE 命令

```
? ■■■■■■■■■■■\n/ ■■■■■ loginUser() ■■■■■■\n! ■■■■■■■■■■■■■\n* ■■■■■ User.c\n* ■■■■ User.c\n* ■■■■ userSave(
```

A.5 任务图

```
main\n ■■■■ userSave\n      ■■■■ userValidate\n      ■■■■ jsonEncode\n      ■■■■ fileWrite
```

A.6 端到端生产流程

```
■■■■■\n    ↓\nT1■■■■■■■■■■■\n    ↓\n■■■■■■■■■■■\n    ↓\nAI ■■■■■■■■■■■■\n    ↓\nT2■■■■■■/■■■■■■■■\n    ↓\nT3■■
```

附录 B｜术语表与核心结论

术语	本文中的含义
AI 原生编程	把 AI 纳入语言表达、函数生产、IDE 交互、上下文管理和工程控制的完整体系
AI::Fun	概念性的 AI 函数生产声明
AI::Struct	概念性的 AI 结构体生产声明
AI::EdgeFun	用于外部与内部边界转换的 AI 生产函数类别
Context Scope	AI 当前任务可见和可操作的信息范围
Task Graph	描述开发任务、依赖、状态与阻塞关系的图
Execution Graph	描述程序运行时调用关系和执行路径的图
TCT	Translation – Transmission – Computation：翻译、传输、计算的结构控制原则
ECL	Engineering Control Loop：观察、决策、执行、验证的工程反馈循环
Controlled Intelligent Production	受控的智能生产：让 AI 的生产能力始终处于明确范围、接口、约束和验证之内

最终总结

> 这不是单纯提高代码生产率的工具，而是一套能够承受 AI 大规模生产能力的工程体系。

传统编译工具链提供确定性执行基础；AI 提供灵活的局部生产能力；上下文管理控制 AI 的可见范围和权限；TCT 提供结构边界；ECL 提供持续反馈；IDE 提供人类与系统之间的统一交互。

真正稀缺的能力不是让 AI
多生成一万行代码，而是让一万个局部生成任务组成一个仍然简单、可靠、可维护的系统。

> 核心理念：受控的智能生产（Controlled Intelligent Production）。

说明：本文为讨论内容的系统化整理，保留了提出过的设计构想、术语、交互约定、工程原则和待验证问题。文中的流程和示例用于表达设计意图，仍需通过原型、用户测试和工程实践验证。